CS3210: Operating Systems

Lecture 4 — OS Abstractions: Isolation

John Kim
Georgia Institute of Technology

January 27, 2026

Contents

1 Administrative Notes **3**

2 What Is Isolation? **3**

3 The Case For Isolation: Benefits **3**

4 The Cost of Isolation: Why Full Isolation Is Useless **4**

5 When Is Isolation Needed? **4**

6 Isolation Tradeoffs **4**

7 What Might We Want to Isolate? **5**

8 Isolation Mechanisms **5**

9 Hardware Isolation: Protection Rings **6**
9.1 The x86 Ring Model . . . 6
9.2 What Ring 0 Protects . . . 6
9.3 The Segment Selector and Current Privilege Level . . . 6
9.4 The Code-Segment Descriptor . . . 7
9.5 CPL in xv6: The GDT Setup . . . 7
9.6 Segment Selectors and Interrupt Handling . . . 8

10 The Unit of Isolation: The Process **8**

11 Memory Isolation via Virtual Memory **9**
11.1 The Virtual Address Space Abstraction . . . 9
11.2 PDE and PTE Permission Flags in xv6 . . . 9

12 CPU Isolation via Time-Slicing **10**
12.1 The Scheduling Abstraction . . . 10
12.2 CPU Isolation Concerns . . . 10
12.3 Time-Slicing in xv6: The Scheduler Loop . . . 10

13 The System Call Interface **11**
13.1 Why a Controlled Handover? 11
13.2 The Trap Handler's Responsibilities 11
13.3 Why Argument Sanitization Matters: The Virus Scenario 11
13.4 Making System Calls in xv6 12
13.5 Trap Numbers in xv6 12
13.6 Scale of System Call Interfaces 13

14 Isolation: A Solved Problem? **13**

15 Summary **14**

16 References **15**

1 Administrative Notes

Lab 0 closed on Friday, January 23 with 148 submissions. If you submitted, expect to see your grade on Gradescope shortly.

Lab 1 (Startup, CPU Stacks, Debugging, and Physical Memory) was assigned on January 23 and is due **February 6, 2026**. Lab 1 must be completed *individually*—no partners for this assignment. Use the full two weeks; the autograder caps daily feedback submissions, so starting early is essential.

Starting with **Lab 2**, teams of 1–2 are permitted and encouraged. If you want a partner, use the "Search for Teammates!" post pinned on Piazza—state your background, availability, specialisation, goals, and the times of week you can pair-program. Student groups will be created under Canvas → People once teams are finalised. Partner changes are only allowed *between* Labs 2, 3, and 4; once a lab begins you are committed to your current partner until that lab closes.

2 What Is Isolation?

The word *isolation* in everyday English means to set or place something apart—to detach or separate it so that it stands alone. In the context of operating systems the same intuition applies: **isolation** is the property by which one software entity (a process, a user, or the kernel itself) is protected from the faults, misbehaviour, or malicious intent of other entities sharing the same hardware.

This lecture develops that concept from first principles: what isolation gives us, what it costs, when it is appropriate, and crucially, how an OS actually implements it on real hardware.

3 The Case For Isolation: Benefits

Isolation serves three distinct and equally important purposes in OS design.

Failure isolation is the most fundamental. If one process crashes, scribbles over memory, or enters an infinite loop, the damage must be contained. It must not cascade into other processes, and above all it must not destroy the OS itself. This is one of the primary motivations for the entire architectural split between user mode and kernel mode. Prof. Tumanov will return to failure isolation extensively throughout the semester because it is the single most important driver of kernel structure and design decisions.

Performance isolation ensures that one process's misbehavior cannot starve others. A process that consumes all available CPU time, allocates every page of physical memory, or holds disk I/O bandwidth hostage must be constrained. This isolation dimension raises a fundamental policy question: should the OS offer *best-effort* fairness (trying to be fair but with no hard guarantees) or *guaranteed* fairness (each process receives at least a specified minimum share)? The answer depends on the workload and is a core scheduling design decision.

Correctness isolation guarantees that the behaviour of one program is not affected by the presence or activity of other programs running concurrently. Even if process B is doing nothing harmful—it is just running—process A's outputs should be identical to what they would be if A were the only program on the machine. Without correctness isolation, bugs in process B can corrupt shared data structures or interfere with process A's memory in subtle ways that are extremely difficult to diagnose.

4 The Cost of Isolation: Why Full Isolation Is Useless

Isolation has a dark side. Taken to its logical extreme, *perfect* isolation makes a process completely useless. A perfectly isolated process cannot communicate with anything: it cannot write to the screen, receive keyboard input, send network packets, or even return an exit code. The only scenario in which full isolation is acceptable is when the computation is entirely self-contained—essentially a calculator that takes no input and produces no observable output. That is almost never what we want.

The deeper insight is that isolation is not a binary property. The useful design space lies between the two extremes. The key takeaway from this discussion is:

> We want *some* isolation *sometimes*. How much isolation is appropriate depends on the workload, the trust relationships between entities, and the current phase of execution. These requirements can change dynamically.

5 When Is Isolation Needed?

Two conditions together indicate that isolation is appropriate:

1. **Shared resources.** Isolation is relevant whenever multiple entities access the same underlying physical resource. Physical memory is the canonical example: every process on a multiprocess OS is competing for the same finite pool of DRAM pages. Without isolation, process A can read or overwrite any physical address, including the memory of process B or the kernel itself.

2. **Non-cooperative entities.** Isolation is especially important when the entities sharing a resource are not designed to cooperate—they are written by different authors, have different goals, and may not trust each other. Processes are the classic example of non-cooperative entities. Threads within a single process, by contrast, *share* the same address space intentionally and are presumed to cooperate; imposing strict memory isolation between threads would defeat the purpose of threading.

A useful diagnostic framework is to ask: *What are the entities? What is the shared resource?* Applying this framework reveals whether isolation is needed and at what granularity it should be enforced.

6 Isolation Tradeoffs

Increasing isolation strength moves safety and usability in opposite directions:

Property	**Weaker Isolation**	**Stronger Isolation**
Safety	Worse	Better
Usability	Better	Worse
Performance	Unpredictable	Unpredictable

The safety/usability tradeoff traces a *Pareto frontier*: any point on the frontier is a legitimate design choice; the goal of good OS design is to push that frontier outward toward higher safety *and* higher usability simultaneously—that is the "dream" that active research aims for. A bad design achieves neither safety nor usability—it sits below the frontier through unnecessary overhead or unnecessary

restrictions. The two ends of the current frontier are *Reality 1* (high safety, low usability, like a highly locked-down microkernel) and *Reality 2* (low safety, high usability, like early MS-DOS).

The performance axis is marked with a question mark in the lecture because the relationship is genuinely ambiguous: isolation mechanisms such as paging add overhead (TLB misses, mode switches), but isolation can also *improve* performance by preventing resource contention and enabling better scheduling decisions.

7 What Might We Want to Isolate?

Armed with the necessity conditions and the tradeoff framework, we can enumerate concrete isolation targets:

- **Kernel space from user space** — the kernel manages the hardware and enforces all the other isolation guarantees; if user code could freely overwrite kernel memory, every other isolation mechanism collapses. This is the highest-priority isolation boundary in any OS.
- **User processes from each other** — no process should be able to read another process's memory, monopolise the CPU, exhaust shared file descriptors, or otherwise interfere with a co-resident process's operation.
- **Memory regions** — even within the kernel, different regions (BIOS area, kernel text, kernel heap, per-CPU data) should have appropriate access controls so that a kernel bug does not inadvertently overwrite interrupt vectors or code pages.
- **Security-sensitive code from ordinary code** — cryptographic key material, authentication logic, and other security-critical functions should be isolated from general application code that might be compromised or buggy.

8 Isolation Mechanisms

Modern operating systems on x86 hardware use five principal mechanisms to achieve isolation. These mechanisms are layered and complementary; each addresses a different dimension of the isolation problem.

1. **Hardware protection rings** — the CPU enforces privilege levels that restrict which instructions and which resources are accessible at each level.
2. **Address spaces** — virtual memory with page mapping gives every process its own isolated view of memory, preventing cross-process address access.
3. **Time-slicing** — the scheduler allocates CPU quanta to processes, ensuring no process can monopolise the processor.
4. **System call interface** — a controlled, vetted gateway through which user code can invoke kernel services without exposing kernel internals.
5. **Permissions and privilege levels** — UNIX-style user/group/other file permissions and UID/GID separation between users.

The remainder of this lecture examines each mechanism in depth with concrete xv6 examples.

9 Hardware Isolation: Protection Rings

9.1 The x86 Ring Model

The x86 architecture defines four **protection rings** (privilege levels 0 through 3). Ring 0 is the most privileged; Ring 3 is the least privileged. The original design allocated different rings to different software layers:

Ring	Privilege Level	Intended Occupant
0	Highest	Operating System Kernel
1	High	OS Services
2	Medium	OS Services
3	Lowest	User Applications

In practice, virtually all modern operating systems—including xv6, Linux, and Windows—use only rings 0 and 3. Rings 1 and 2 were intended for OS services that need some privilege but not full kernel access; this design was never widely adopted because the complexity of managing three distinct privilege levels outweighed the benefits. The resulting binary "kernel mode / user mode" model is simple, robust, and sufficient.

9.2 What Ring 0 Protects

Running in ring 0 (also called **kernel mode**) grants access to CPU resources that are explicitly protected from ring 3:

- **Writes to `%cr3`** — CR3 holds the base physical address of the page directory. A process that could write `%cr3` could replace its own page table with any other, instantly gaining access to any physical memory. Ring 0 protection of CR3 is therefore the foundation of all address-space isolation.
- **Writes to `%cs`** — the low two bits of `%cs` encode the **Current Privilege Level (CPL)**. If user code could write `%cs` directly it could elevate itself to ring 0, bypassing all privilege checks.
- **Every memory read/write is checked against the privilege level** — page table entries carry a **U/S** (User/Supervisor) bit. When CPL = 3 and the U/S bit of the target page is 0 (supervisor-only), the hardware raises a general protection fault. This is the primary mechanism preventing user code from reading or writing kernel memory.
- **I/O port accesses are privileged** — the `IN` and `OUT` instructions that communicate with hardware devices (including disk controllers) are restricted to ring 0. An unprivileged process that could issue arbitrary `OUT` commands could overwrite disk sectors, corrupt filesystems, or reprogram the interrupt controller.
- **Control register accesses** — writes to EFLAGS, CR4, and other control registers that govern CPU operating mode are ring 0-only.

9.3 The Segment Selector and Current Privilege Level

The CPU knows the currently executing privilege level through the **segment selector** stored in `%cs`. A segment selector is a 16-bit value with the following structure:

- **Bits 15–3 (Index)** — an index into the GDT or LDT.
- **Bit 2 (TI, Table Indicator)** — 0 selects the GDT; 1 selects the LDT.
- **Bits 1–0 (RPL, Requested Privilege Level)** — the privilege level being requested for this segment reference.

The **CPL** is defined as the two least-significant bits of `%cs`. When CPL = 0 the processor is in ring 0 (kernel mode); when CPL = 3 it is in ring 3 (user mode). When the kernel switches to user mode it sets CPL = 3 by loading a user-mode code segment selector; when it takes a trap or interrupt it restores CPL = 0 by loading a kernel-mode selector. The CPU itself enforces that ring 3 code cannot raise CPL by writing `%cs` directly—only the hardware trap mechanism may change CPL.

9.4 The Code-Segment Descriptor

A segment descriptor is an 8-byte entry in the GDT that describes the segment's base, limit, and permission flags. For a code segment the relevant flags are: **P** (Present, must be 1 for a valid segment), **DPL** (Descriptor Privilege Level, 2 bits—the minimum CPL required to use this segment), **G** (Granularity—0 means the limit is in bytes; 1 means pages), **D/B** (Default operand size, 1 for 32-bit), **L** (Long mode bit), **C** (Conforming), **R** (Readable), and **A** (Accessed).

The DPL field is the key isolation lever: a user-mode code segment has DPL = 3, while the kernel's code segment has DPL = 0. As a concrete example, the value `0x009b` in `%cs` under QEMU decodes as:

$$\texttt{0x009b} = 0000\ 0000\ 1001\ 1011_2$$

Bits 15–3 give a GDT index; bits 1–0 = 11_2 = 3, confirming CPL = 3 (user mode).

9.5 CPL in xv6: The GDT Setup

xv6 maintains a per-CPU GDT configured in two stages. The **bootstrap GDT** (in `bootasm.S`) is minimal—just a null entry, a flat code segment, and a flat data segment—sufficient to get into protected mode. Once the main kernel initialises each CPU, it replaces this with a more complete per-CPU GDT:

Listing 1: vm.c — per-CPU GDT initialisation showing DPL distinction

```
c = &cpus[cpunum()];
c->gdt[SEG_KCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, 0);         // DPL=0
c->gdt[SEG_KDATA] = SEG(STA_W,       0, 0xffffffff, 0);         // DPL=0
c->gdt[SEG_UCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, DPL_USER); // DPL=3
c->gdt[SEG_UDATA] = SEG(STA_W,       0, 0xffffffff, DPL_USER); // DPL=3
// Per-CPU private data pointer in GS
c->gdt[SEG_KCPU]  = SEG(STA_W, &c->cpu, 8, 0);

lgdt(c->gdt, sizeof(c->gdt));
loadgs(SEG_KCPU << 3);
```

The four `SEG(...)` calls create flat segments (base = 0, limit = `0xffffffff`) that span the entire 4 GB address space. The isolation does *not* come from segment limits—all segments are identical in

base and limit. It comes from the **DPL**: kernel code and data segments carry DPL = 0, while user code and data segments carry DPL = 3 (`DPL_USER`). The CPU prevents ring 3 code from loading a ring 0 segment selector into `%cs`, so user code is confined to the user code and data segments and cannot access kernel-privilege instructions.

9.6 Segment Selectors and Interrupt Handling

The segment selector mechanism also governs who may invoke which interrupt handlers. Each entry in the **Interrupt Descriptor Table (IDT)** specifies the target `CS` and `EIP` for the handler, plus a DPL in the gate descriptor. When an interrupt fires, the CPU checks whether CPL $\leq$ DPL of the gate descriptor:

- If CPL $\leq$ DPL, the interrupt is permitted and the handler runs.
- If CPL $>$ DPL (user trying to fire a ring 0-only gate), the CPU raises a general protection fault.

This mechanism ensures that user code cannot directly invoke privileged interrupt handlers—the only valid entry point into the kernel is through approved gates in the IDT.

10 The Unit of Isolation: The Process

The central unit of isolation in a traditional OS is the **process**. Four goals follow from this design choice:

1. **Process-to-process isolation** — process X must not be able to read or write process Y's memory, exhaust Y's file descriptors, or consume CPU in a way that starves Y. This applies to memory, CPU time, open file descriptors, and any other limited resource.
2. **Process-to-kernel isolation** — a process must not be able to disable or circumvent the kernel's enforcement of isolation for other processes. If a process could crash the kernel, the protection guarantees for every other process collapse instantly.
3. **Robustness against bugs and malice** — isolation must hold even if a process contains bugs (which is inevitable) or is deliberately adversarial (which must be assumed). The infamous **Spectre** and **Meltdown** hardware vulnerabilities demonstrate that even well-designed software isolation can be undermined by hardware-level attacks that exploit speculative execution; these attacks are discussed in Section 14.
4. **Failure containment** — if process A has a bug, that bug must not propagate to process B unless B is a child of A and the relationship was explicitly established. Independent processes must be truly independent.

A subtle and interesting question: can we isolate a process *from the kernel itself*? Normally the kernel is trusted. But the research on microkernels and capability-based systems explores exactly this question, attempting to build OSes where even kernel components are isolated from each other.

11 Memory Isolation via Virtual Memory

11.1 The Virtual Address Space Abstraction

Virtual memory is the primary mechanism for memory isolation. The core idea is to give every process the illusion that it has its own private address space spanning the entire addressable range of the CPU—on 32-bit x86, that is 4 GB from address `0x00000000` to `0xFFFFFFFF`—while the OS uses the hardware **Memory Management Unit (MMU)** to map each virtual address to a distinct physical address.

Because each process has its own page table (pointed to by CR3), a virtual address `0x10000` in process A and the same virtual address `0x10000` in process B map to *completely different physical pages*. Process A cannot read process B's memory by guessing virtual addresses, because A's page table simply does not contain mappings to B's physical pages. This gives *address space isolation* automatically: the mapping hardware enforces it without any explicit per-access check by the kernel.

In xv6, the virtual address space is divided at `0x80000000`:

- **Below** `0x80000000`: user space. Contains the user text and data, heap, and user stack.
- **Above** `0x80000000`: kernel space. The kernel's text and data (beginning at `0x80100000`) and free memory are mapped here, with the BIOS area placed at `0x80000000`.

The kernel mappings are *present in every process's page table*—the same physical kernel pages are mapped at the same high virtual addresses in all processes. This allows the kernel to run using the same virtual addresses regardless of which process is currently active. However, the kernel pages are marked *supervisor-only* (U/S = 0), so user code that tries to dereference a kernel address gets a page fault rather than a successful read.

11.2 PDE and PTE Permission Flags in xv6

The x86 paging hardware provides two bits in every page-directory entry (PDE) and page-table entry (PTE) that implement per-page isolation:

- **U/S (User/Supervisor, bit 2)** — 0 means the page is accessible only from ring 0 (supervisor mode); 1 means any privilege level may access the page. Kernel pages are mapped U/S = 0; user pages are mapped U/S = 1.
- **R/W (Read/Write, bit 1)** — 0 makes the page read-only; 1 allows writes. Code pages are typically R/W = 0 (to catch accidental code overwrite and to support copy-on-write); data pages are R/W = 1.
- **P (Present, bit 0)** — 1 means the page is valid; 0 causes a page fault on any access. The OS uses P = 0 for pages that have been swapped to disk or have not yet been allocated.

These bits are checked *at every memory access* by the MMU hardware, with no software involvement unless a fault occurs. The combination of separate page tables per process (for coarse isolation) and U/S flags within the kernel's shared mappings (for fine-grained privilege enforcement) gives xv6 a layered, robust memory isolation model.

12 CPU Isolation via Time-Slicing

12.1 The Scheduling Abstraction

While virtual memory isolates the *memory* dimension, the CPU must also be shared fairly and safely. The OS implements **time-slicing**: each process is given a short *time quantum* on the CPU, after which the scheduler preempts it and runs the next runnable process. From each process's perspective, it appears to have a dedicated CPU—the scheduling abstraction creates the *illusion of parallelism*.

A **scheduler** answers three questions: *who* gets the CPU, *what* (which process), and *when*. The policy choice between **cooperative scheduling** (processes voluntarily yield with a `yield()` call) and **preemptive scheduling** (a timer interrupt forces the switch) is critical for isolation: cooperative scheduling breaks down if any process is buggy or adversarial and never yields, whereas preemptive scheduling guarantees that no process can monopolise the CPU indefinitely.

12.2 CPU Isolation Concerns

Achieving perfect CPU isolation is harder than it might appear. Even with time-slicing, processes can interact through *shared CPU state*: cache residency, TLB state, and branch predictor state are all shared resources that time-slicing does not isolate. These shared microarchitectural resources are precisely the attack surface exploited by Spectre and Meltdown. For the purposes of the course, however, the scheduler model treats the CPU as fully re-initialised between quanta.

12.3 Time-Slicing in xv6: The Scheduler Loop

xv6's scheduler is a simple round-robin loop that runs on each CPU:

Listing 2: proc.c — the per-CPU scheduler loop

```
for(;;){
    sti();  // Enable interrupts so timer can fire

    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->state != RUNNABLE)
            continue;

        // Switch to chosen process.
        // It is the process's job to release ptable.lock
        // and then reacquire it before jumping back to us.
        proc = p;
        switchuvm(p);           // load process p's page table
        p->state = RUNNING;
        swtch(&cpu->scheduler, p->context);  // context switch
        switchkvm();            // restore kernel page table

        // Process is done running for now.
        proc = 0;
    }
    release(&ptable.lock);
}
```

Three isolation-relevant operations occur inside the loop. First, `switchuvm(p)` loads CR3 with the physical address of process p's page directory, instantly replacing the entire address space. Second, `swtch()` saves the current CPU context (register values) and restores the saved context of process p, transferring EIP to wherever p last left off. Third, after the process yields or is preempted and returns to the scheduler, `switchkvm()` restores the kernel-only page table, ensuring that when the scheduler code executes it has a clean kernel view of memory.

13 The System Call Interface

13.1 Why a Controlled Handover?

User code frequently needs kernel services: reading from a file, forking a child process, allocating memory. These operations require ring 0 privileges, but user code runs at ring 3. The transition from ring 3 to ring 0 must be *controlled*—if user code could jump to an arbitrary kernel address, it could bypass all argument validation and execute kernel code with full privilege in an unexpected state, compromising isolation entirely.

The solution is the **trap handler**: a fixed, predetermined entry point into the kernel that the hardware enforces. User code cannot jump to any kernel address; it can only trigger a trap through the approved mechanism.

13.2 The Trap Handler's Responsibilities

When user code invokes a system call, the trap handler must perform a well-defined sequence before executing any kernel logic:

1. **Set aside the user-space context** — save all registers onto the kernel stack so that the user context can be restored exactly on return.
2. **Load a safe kernel context** — switch to the kernel stack and kernel data structures. At this point the CPU is in ring 0 with the kernel's view of memory.
3. **Sanitize arguments** — validate every pointer and value passed by the user. Critically, any pointer argument must be checked to confirm it lies within user space. A user process must not be able to pass a kernel pointer as an argument to a system call and thereby trick the kernel into reading or writing kernel memory on its behalf.
4. **Accept or deny the request** — check permissions (does this process have access to this file?), quota limits, and any other policy requirements. Only after all checks pass does the kernel execute the requested operation.

13.3 Why Argument Sanitization Matters: The Virus Scenario

The following code illustrates what happens when argument sanitization is absent:

Listing 3: Unsanitized system call argument enables kernel memory overwrite

```
char *buff = get_virus_code();
int fd = open("/tmp/virus", ...);
write(fd, buff, size);                          // write virus to disk
read(fd, (char *)(0x80100000), size);           // CHECK NEEDED HERE
```

The address `0x80100000` is the start of the xv6 kernel's text segment—kernel code itself. If the `read()` system call does not verify that the destination pointer lies within user space, the kernel will happily copy the virus payload from the file directly into its own code pages, overwriting the instruction stream it is currently executing. The diagram in the lecture slide shows the virus code in the user heap and also overwriting the kernel text, connected by the blue circular arrow that represents this attack path.

A correct implementation checks that the destination address passed to `read()` falls below the kernel boundary (`0x80000000` in xv6) before copying any data. This single check prevents an entire class of kernel-overwrite attacks.

13.4 Making System Calls in xv6

On the user side, every system call is generated by a macro that follows the x86 calling convention for software interrupts:

Listing 4: usys.S — SYSCALL macro used to generate all xv6 system call stubs

```
#define SYSCALL(name)      \
  .globl name;             \
  name:                    \
      movl $SYS_##name, %eax;   \  /* put syscall number in EAX */
      int  $T_SYSCALL;           \  /* fire INT 64               */
      ret

SYSCALL(fork)
SYSCALL(exit)
```

Each system call stub places the system call number in `%eax` (xv6's convention), then executes `INT $64` (`T_SYSCALL = 64`). The `INT` instruction atomically: saves EIP, CS, and EFLAGS onto the kernel stack; loads the CS and EIP from the IDT gate for vector 64; and sets CPL = DPL of the gate (0 for the syscall gate). This is the single, hardware-enforced transition from ring 3 to ring 0. The `ret` instruction at the end of the stub is executed *after* the kernel returns from handling the system call and `IRET` restores the user context.

13.5 Trap Numbers in xv6

xv6 defines its trap numbers in `traps.h`. The first 20 entries are processor-defined exceptions; xv6 adds its own starting at number 64 to avoid conflicts:

Listing 5: traps.h — processor-defined and xv6-defined trap numbers

```
// Processor-defined:
#define T_DIVIDE   0  // divide error
#define T_DEBUG    1  // debug exception
#define T_NMI      2  // non-maskable interrupt
#define T_BRKPT    3  // breakpoint
#define T_OFLOW    4  // overflow
#define T_BOUND    5  // bounds check
#define T_ILLOP    6  // illegal opcode
#define T_DEVICE   7  // device not available
#define T_DBLFLT   8  // double fault
#define T_TSS     10  // invalid task switch segment
```

```
#define T_SEGNP   11  // segment not present
#define T_STACK   12  // stack exception
#define T_GPFLT   13  // general protection fault
#define T_PGFLT   14  // page fault
#define T_FPERR   16  // floating point error
#define T_ALIGN   17  // alignment check
#define T_MCHK    18  // machine check
#define T_SIMDERR 19  // SIMD floating point error

// xv6-defined (chosen to not overlap processor-defined):
#define T_SYSCALL 64  // system call
#define T_DEFAULT 500 // catchall
```

The choice of 64 for `T_SYSCALL` is arbitrary but deliberate: it must be above the processor's reserved range (0–31), and it must not conflict with any hardware interrupt that has been mapped by the PIC (Programmable Interrupt Controller) into the IDT. The `T_DEFAULT` catchall at 500 exists so that unexpected interrupts do not silently execute garbage code.

13.6 Scale of System Call Interfaces

The system call table is the entire visible surface area between user code and the kernel. The size of this table is a design choice that reflects the OS's philosophy:

OS	System Calls
xv6	21
Linux	300+
FreeBSD	500+
Windows 10–11	≈465 core / 2000+ total

xv6's minimalist table of 21 syscalls is intentional: the course uses xv6 precisely because it implements all core OS mechanisms without the complexity of a production system. A useful debugging tool on Linux is `strace`, which traces every system call made by a process—it is invaluable for understanding what a program is actually doing at the OS boundary.

14 Isolation: A Solved Problem?

A natural question after seeing all the isolation mechanisms above is: is isolation a solved problem? The answer, unfortunately, is no. Two landmark security vulnerabilities disclosed in January 2018 demonstrated that hardware-level attacks can break the isolation guarantees described in this lecture, even when the software is perfectly correct.

Meltdown exploits the fact that modern processors speculatively execute instructions before checking whether those instructions are permitted. A user process can cause the CPU to speculatively load a kernel memory address (which would ordinarily be blocked by the U/S page-table bit), and before the fault is raised, use the speculatively loaded value to encode information into the CPU cache via a carefully crafted cache-timing side channel. By subsequently measuring cache access times, the attacker recovers the kernel memory contents without ever having architectural access to them. Meltdown was described by its discoverers as something that "breaks the most fundamental isolation between user applications and the operating system."

Spectre exploits branch prediction and speculative execution to cause a victim process to speculatively access its own memory in a way that leaves a cache-timing side channel readable by an attacker in a different process. Unlike Meltdown, Spectre does not require accessing kernel memory; it works across process boundaries. Its discoverers characterised it as breaking "the isolation between different applications."

Both attacks are deeply rooted in microarchitectural optimisations that were designed for performance, not isolation. They underscore the lecture's key point about the safety/usability tradeoff and the "performance" axis being a question mark: the very optimisations that make modern CPUs fast (speculative execution, out-of-order execution, branch prediction) create new attack surfaces that can erode even well-designed software isolation. Mitigations (KPTI, retpoline, microcode updates) have been deployed, but they come at measurable performance cost and do not fully eliminate the underlying problem.

15 Summary

- **Isolation definition**: the property by which an OS entity is protected from the faults, misbehaviour, or malice of other entities sharing the same hardware.
- **Benefits**: failure isolation (faults don't propagate), performance isolation (no starvation), correctness isolation (results unaffected by other programs).
- **Costs**: 100% isolation is useless—a process must communicate to be useful. We want *some* isolation *sometimes*, calibrated to workload and trust relationships.
- **Necessary conditions**: isolation is appropriate when resources are shared *and* entities are non-cooperative.
- **Tradeoffs**: stronger isolation → safer but less usable; the goal of OS design is to push the Pareto frontier toward both high safety and high usability.
- **Mechanisms**: hardware protection rings, virtual address spaces, time-slicing, system call interface, and user/group permissions.
- **Protection rings**: x86 has four rings; practical OSes use only rings 0 and 3. CPL (lower 2 bits of `%cs`) determines current privilege. Ring 0 guards CR3, CS writes, memory privilege checks, I/O ports, and control registers.
- **Segment selectors**: encode GDT index, TI (GDT/LDT), and RPL. CPL is derived from the RPL of the loaded `%cs` selector.
- **Memory isolation**: each process gets its own virtual address space via a private page table. U/S and R/W bits in PDE/PTE entries enforce per-page access control at hardware speed.
- **xv6 GDT**: kernel segments have DPL = 0; user segments have DPL = 3 (`DPL_USER`). All four segments are flat (base 0, full 4 GB limit); isolation comes from DPL and from paging, not from segment limits.
- **CPU isolation**: time-slicing via a preemptive scheduler; xv6 implements round-robin using `switchuvm()` + `swtch()` + `switchkvm()`.
- **System call interface**: the only approved ring-3 → ring-0 pathway; user code fires `INT $64`; the trap handler saves context, loads kernel context, *sanitizes arguments*, and accepts or

denies the request.

- **Argument sanitization**: every pointer passed to a syscall must be verified to lie within user space; failure to do so can allow a process to overwrite kernel memory.
- **Limits of isolation**: Meltdown and Spectre (2018) showed that hardware speculative execution can break software isolation at a microarchitectural level, even when all software checks are correct.

16 References

- Tumanov, A. *CS3210: Design of Operating Systems — Lecture 4 Slides.* Georgia Institute of Technology. January 27, 2026.
- Cox, R.; Kaashoek, F.; Morris, R. *xv6: a simple, Unix-like teaching operating system.* MIT CSAIL. `https://pdos.csail.mit.edu/6.828/2022/xv6.html`
- Lipp, M., et al. "Meltdown: Reading Kernel Memory from User Space." *USENIX Security Symposium.* 2018. `https://meltdownattack.com/`
- Kocher, P., et al. "Spectre Attacks: Exploiting Speculative Execution." *IEEE Security & Privacy.* 2019. `https://googleprojectzero.blogspot.com/2018/01/reading-privileged-memory-with-s` `html`
- Boos, K., et al. "Theseus: an Experiment in Operating System Structure and State Management." *USENIX OSDI.* 2020. `https://www.usenix.org/conference/osdi20/presentation/` `boos`
- Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide.* `https://www.intel.com/content/www/us/en/developer/articles/` `technical/intel-sdm.html`
- Silberschatz, A.; Galvin, P. B.; Gagne, G. *Operating System Concepts*, 10th ed. John Wiley & Sons. 2018.

Personal Notes